



Navigation System



Unreal Engine Navigation System

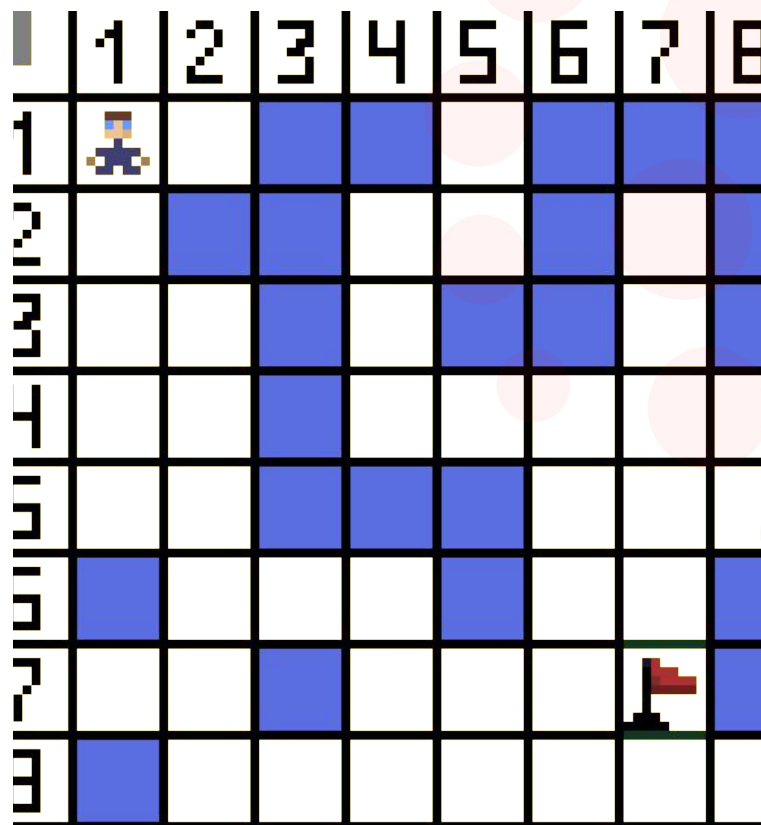
Parliamo del sistema che consente agli agenti controllati dall'AI di muoversi all'interno dell'ambiente virtuale, trovando percorsi ottimali per raggiungere i loro obiettivi.

Questo sistema utilizza tecniche di pathfinding, come A^* o *Dijkstra*, e una rappresentazione spaziale chiamata **nav mesh** (*navigation mesh*), che divide il livello in aree navigabili e ostacoli.



A* per il 3D

A* è ottimale nel caso sia possibile ridurre l'intero spazio di gioco in nodi o celle (strategico a turni, roguelike, RPG, etc).



Nel caso in cui è richiesto un movimento dei personaggi più complesso, A* diventa

- **Troppo pesante** (troppi nodi)
- Oppure **poco realistico** (comunque troppo pochi nodi)



A* per il 3D

I pochi nodi permettono di fare pathfinding in modo efficiente, ma rendono il gioco poco realistico. Inoltre, diventa molto laborioso per i designer piazzare i nodi.



Vorremmo invece definire delle aree in cui i personaggi possono muoversi liberamente, ma che possono usare anche per capire come raggiungere aree adiacenti

Per questo generalmente nel 3D (ed in Unreal Engine di conseguenza) passiamo alle **navigation mesh**.

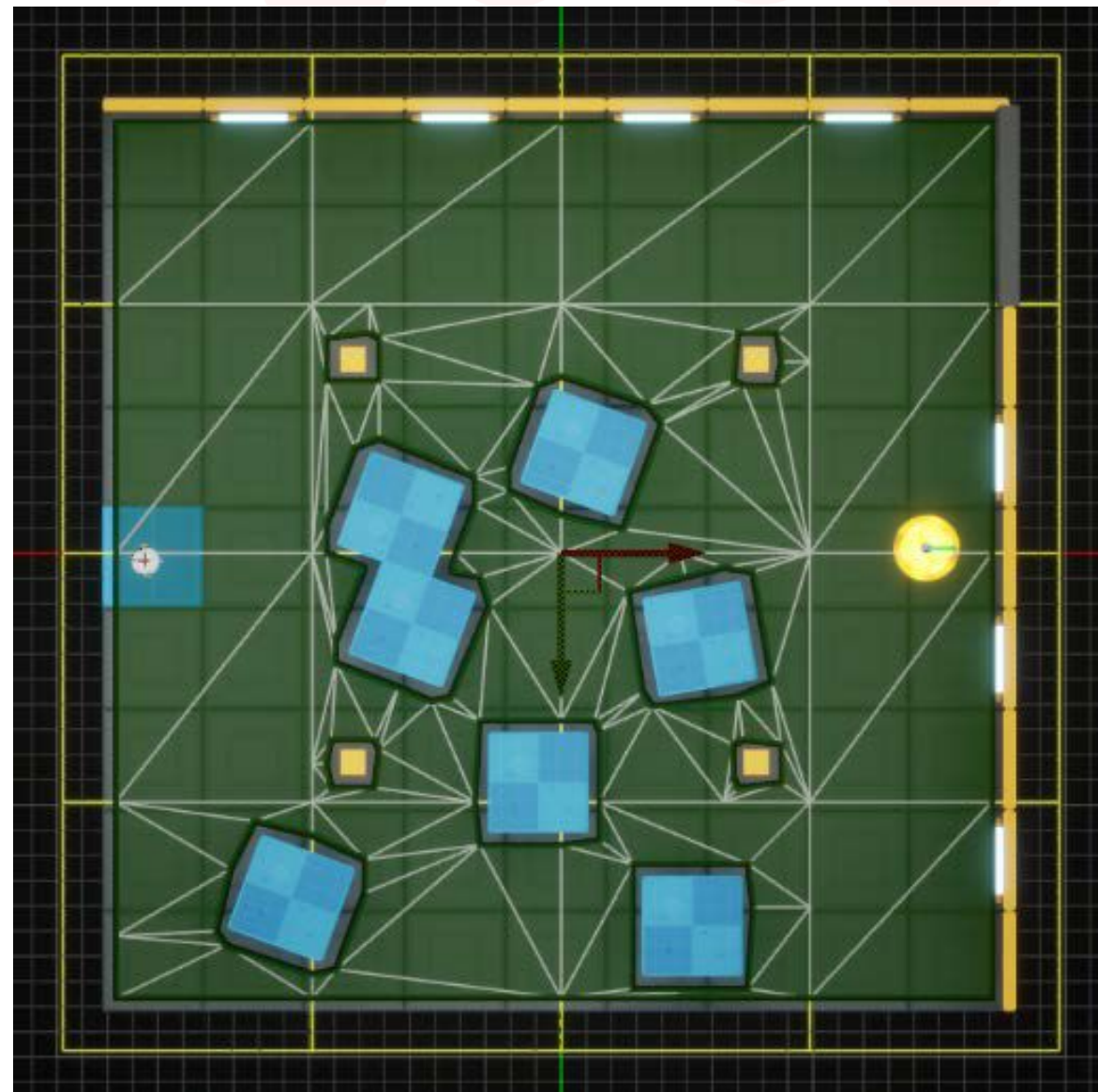


Navigation Mesh

In Unreal Engine, il Navigation System si basa su una navigation mesh che funziona dividendo lo spazio navigabile in poligoni, successivamente suddivisi in **triangoli** per maggiore efficienza. Ogni triangolo viene considerato un nodo di un grafo per raggiungere una posizione specifica e, quando due triangoli sono adiacenti, i rispettivi nodi vengono collegati.

Utilizzando questo grafo, è possibile applicare qualsiasi tipo di algoritmo di pathfinding e il processo risultante genererà un percorso tra questi triangoli che l'AI può attraversare.

Fortunatamente, a meno che non sia strettamente necessario, Unreal Engine utilizza una versione generalizzata dell'algoritmo **A***, rendendolo già ideale per ambienti complessi e dinamici come quello dei videogiochi.



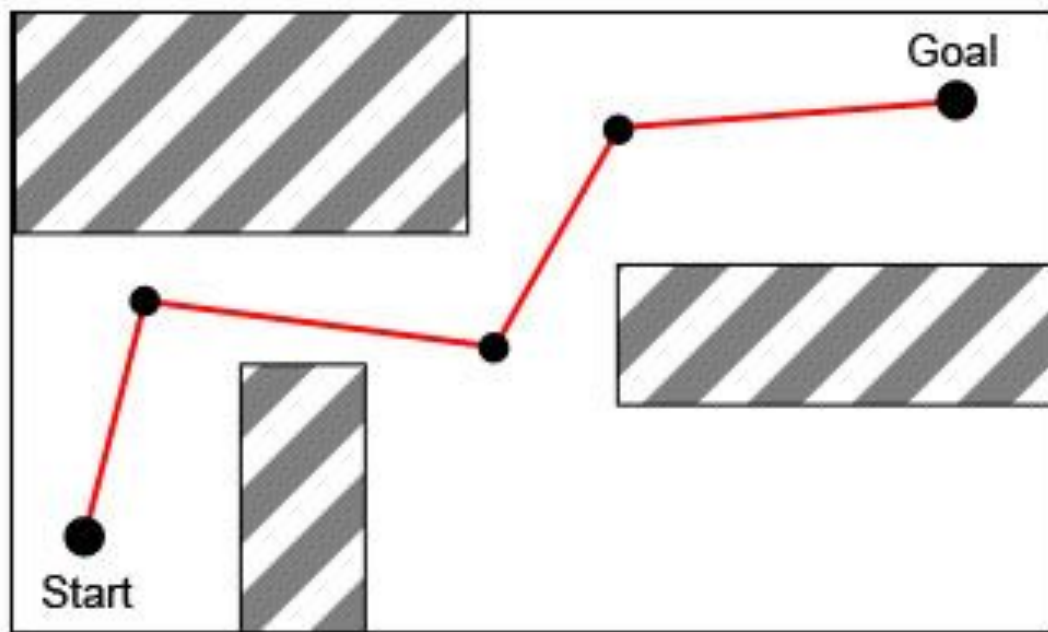


Navigation Mesh

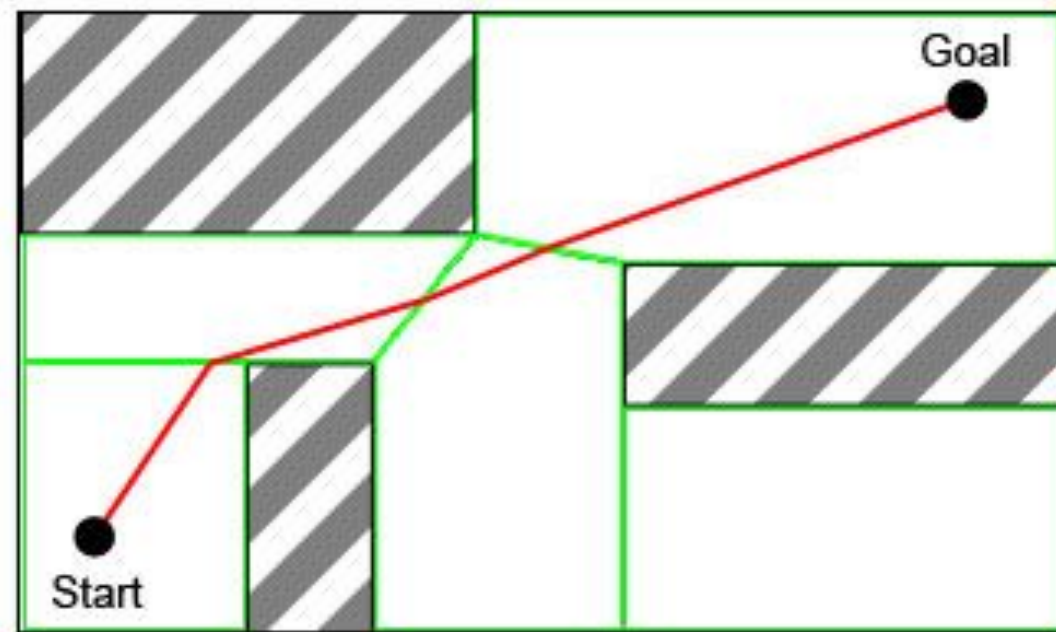
Le navigation mesh (**navmesh**), quindi, ci permettono di definire il movimento all'interno di aree circoscritte (poligoni collegati, perciò mesh).

Ad ogni update, l'agente si muoverà verso il punto visibile lungo il percorso più vicino alla destinazione scelta.

La tecnica funziona molto bene in spazi all'interno dei quali il personaggio può muoversi liberamente.



Waypoints



Navigation Mesh



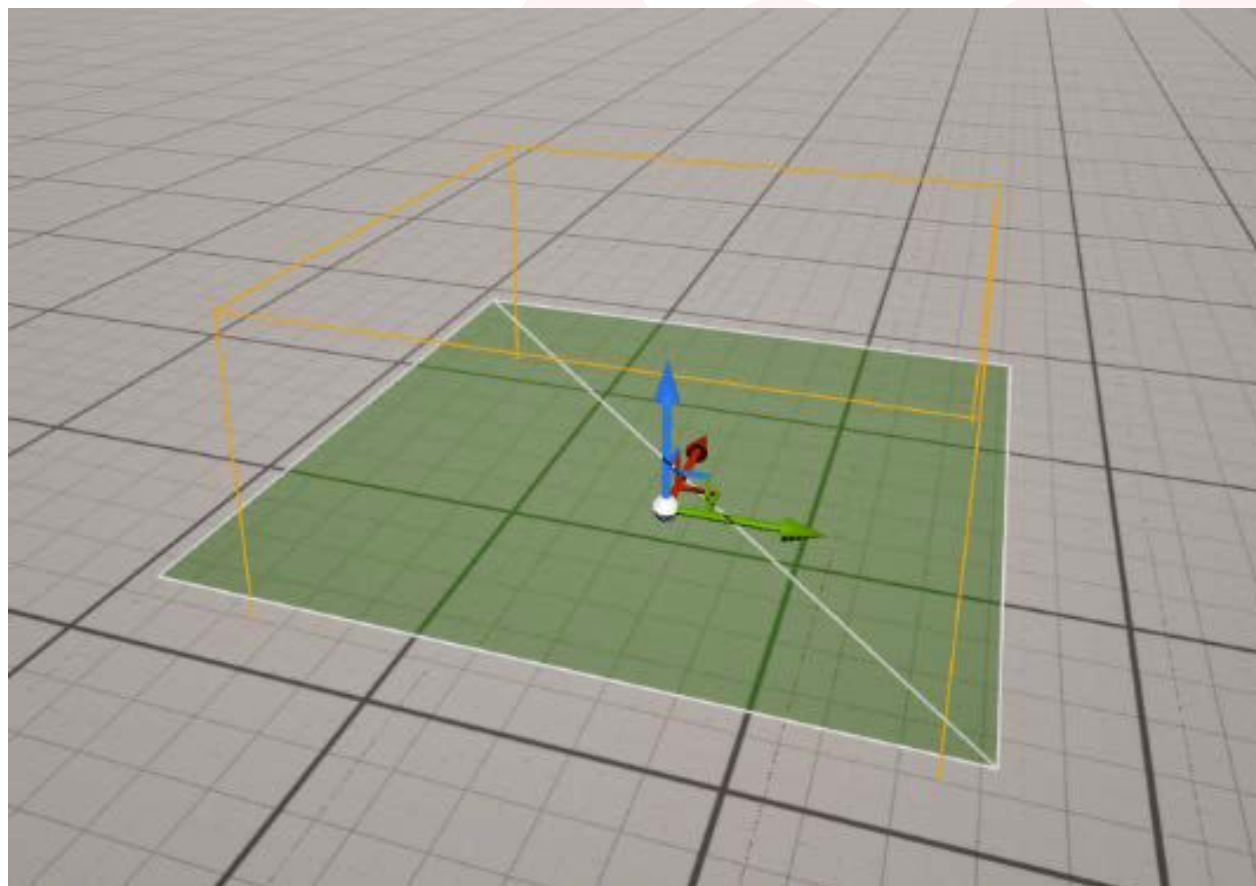
NavMesh in UE5

Per generare una navmesh in Unreal Engine, è sufficiente aggiungere uno o più attori **Nav Mesh Bounds Volume** nel livello e modificarne le dimensioni in base alle proprie esigenze.

La mesh verde, composta da due triangoli, è stata generata da un altro attore: il **Recast Nav Mesh**, che viene solitamente creato automaticamente la prima volta che si aggiunge un Nav Mesh Bounds Volume al livello.

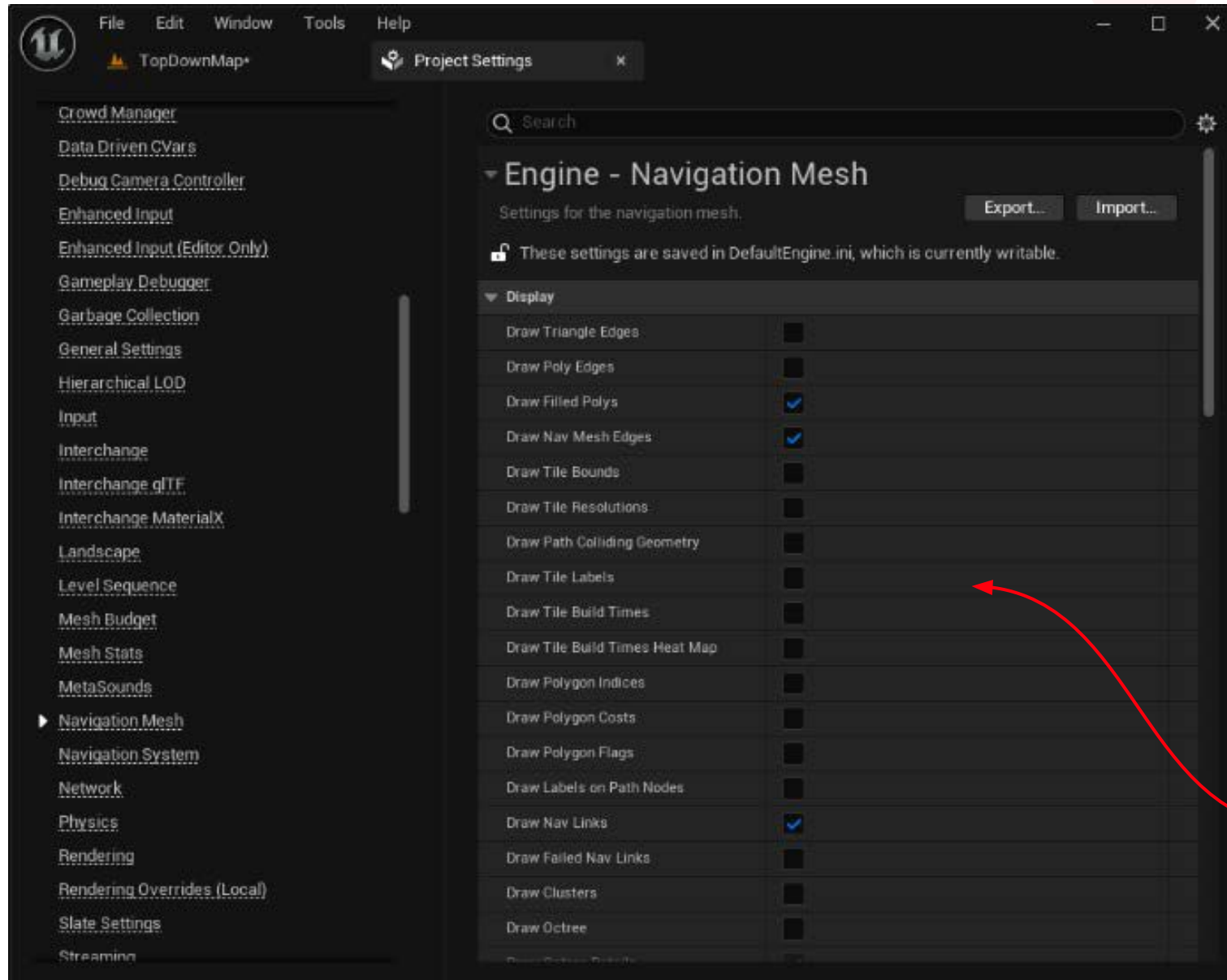
Questo attore si occupa di generare l'area percorribile per le entità AI, che lo utilizzeranno per effettuare calcoli efficienti e precisi sul pathfinding.

Di norma, un'istanza di questo attore viene creata automaticamente non appena si aggiunge un Nav Mesh Bounds Volume al livello.





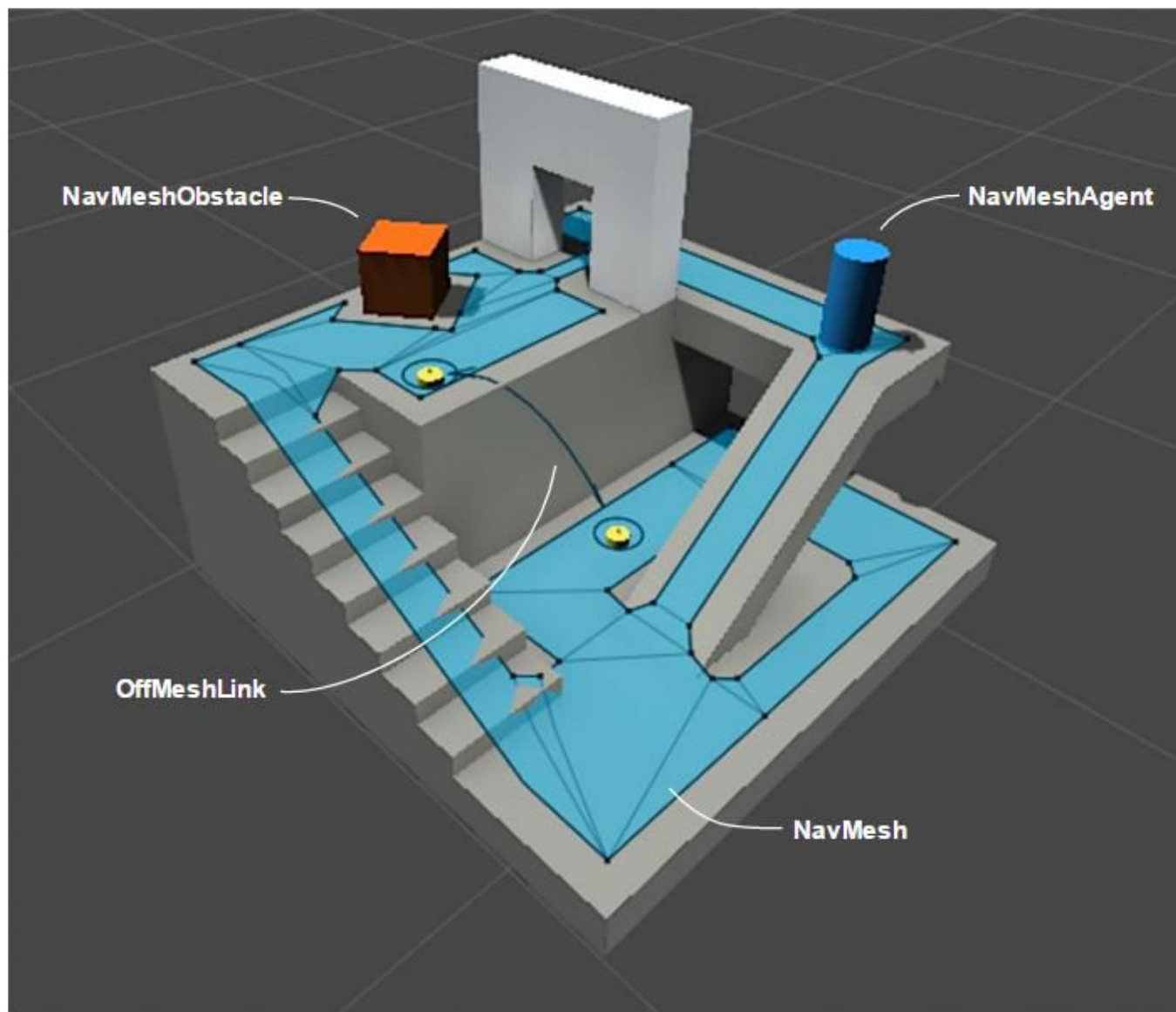
NavMesh in UE5



È importante notare che la maggior parte delle impostazioni disponibili per l'attore **Recast NavMesh** può essere configurata con valori predefiniti nelle Project Settings dell'editor. Queste impostazioni possono essere aperte dal menu File, selezionando la sezione Engine - Navigation Mesh, come mostrato in figura.



Navigation Mesh Components



- **NavMesh:** struttura dati che rappresenta superfici camminabili del mondo sotto forma di poligoni collegati
- **NavMesh Agent:** componente che controlla un personaggio che si muove sulle navmesh.
- **NavMesh Obstacle:** componente che descrive ostacoli sul percorso statici o dinamici
- **Nav Modifier Volume:** componente che definisce volumi per modificare la navmesh
- **Navigation Link:** componente che definisce scorciatoie per la navmesh



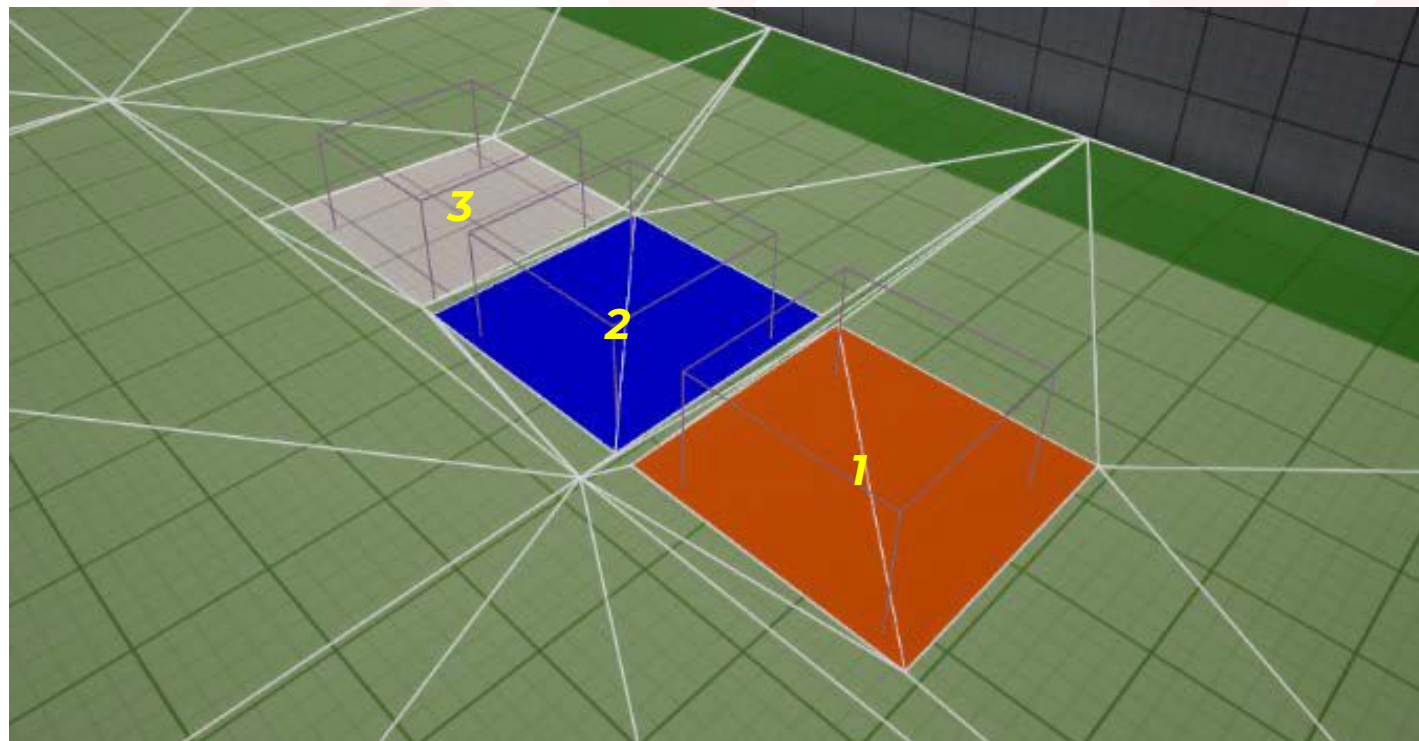
Nav Modifier Volume

Questo attore ha il “semplice” compito di modificare la nav mesh.

Una volta posizionato questo volume nel livello, si ha la possibilità di modificare come un agente AI lo percepirà durante il pathfinding.

Esso potrà essere designato come:

- Un terreno impraticabile **(1)**
- Un terreno difficile **(2)**
- Un ostacolo. **(3)**



È possibile creare un personale **nav modifier** semplicemente estendendo la classe UNavArea, impostarne i parametri e posizionandolo nel livello.

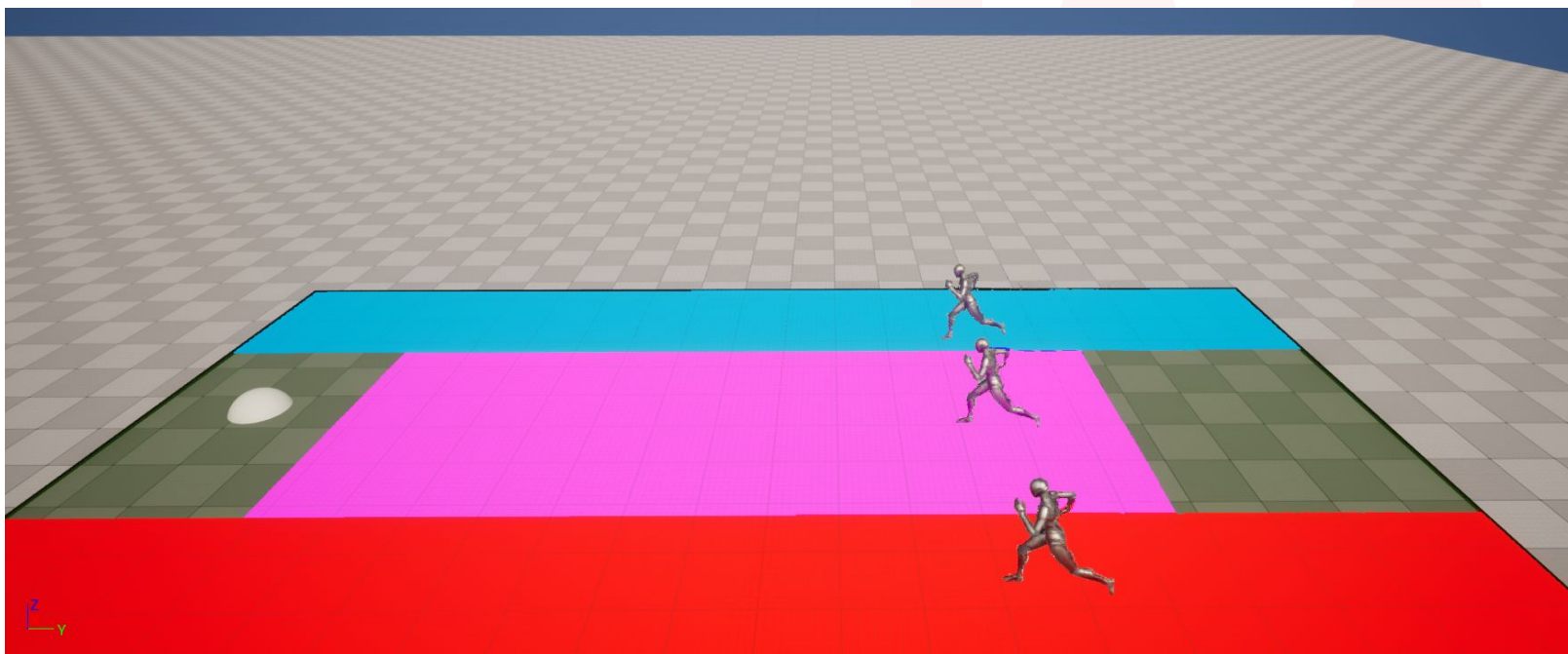


Nav Query Filters

Come metodo aggiuntivo per personalizzare il comportamento del Navigation System durante la generazione dei percorsi per gli agenti AI, puoi sfruttare i **Navigation Query Filters**.

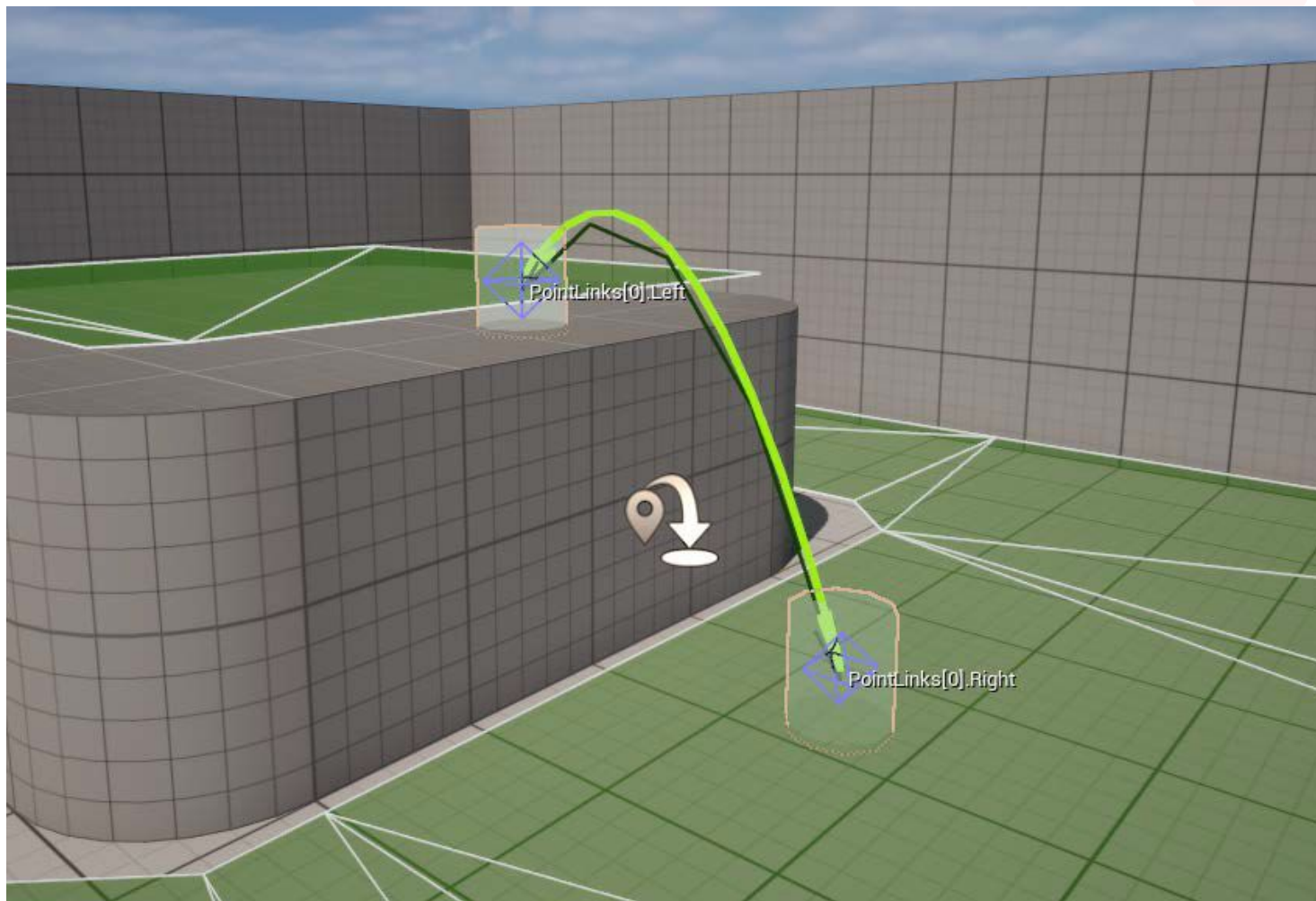
Questo strumento include informazioni relative a una o più aree specifiche e offre la flessibilità di sovrascrivere i valori di costo assegnati alle aree stesse, se necessario.

Implementando i query filter, si ha la possibilità di personalizzare i percorsi di navigazione delle AI mentre attraversano diverse regioni del tuo mondo di gioco, consentendo di affinare e ottimizzare i movimenti degli NPC.





Nav Link Proxies



Durante la progettazione di un terreno percorribile, è comune incontrare spazi vuoti o aree con altitudini diverse.

In questi casi, potrebbe essere necessario che un personaggio AI salti da un lato all'altro.

Proprio per gestire queste situazioni è stato creato il **Nav Link Proxy**, un attore che consente di collegare due aree della nav mesh prive di un percorso diretto di navigazione.

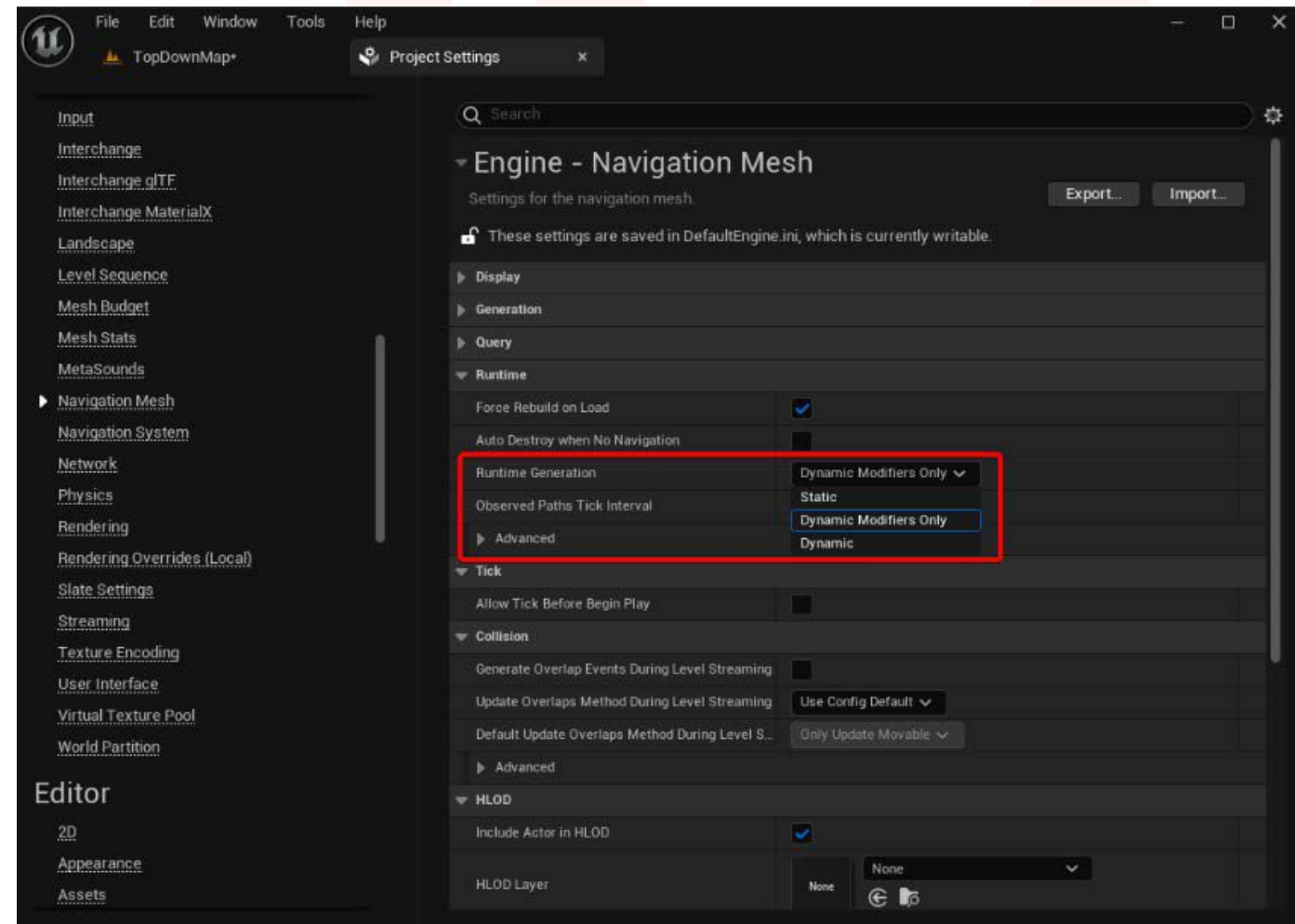
Grazie a questo strumento, è possibile permettere al personaggio di saltare, lasciarsi cadere ed eseguire acrobazie spettacolari, passando fluidamente da una manovra all'altra senza interruzioni.



Nav Mesh Generation

Di default, Unreal Engine è configurato per generare nav mesh in modo **statico**, il che significa che la mesh viene creata offline e non può essere modificata durante il runtime.

Tuttavia, se si necessita di un sistema più flessibile per la generazione della nav mesh, è possibile optare per la generazione **dinamica**, che consente di aggiornare la nav mesh costantemente a runtime in base alle diverse circostanze di gioco, o altrimenti usando la generazione dinamica solo tramite modifiers, ovvero aggiornando la nav mesh a runtime solo tramite l'utilizzo dei Nav Mesh Modifiers.



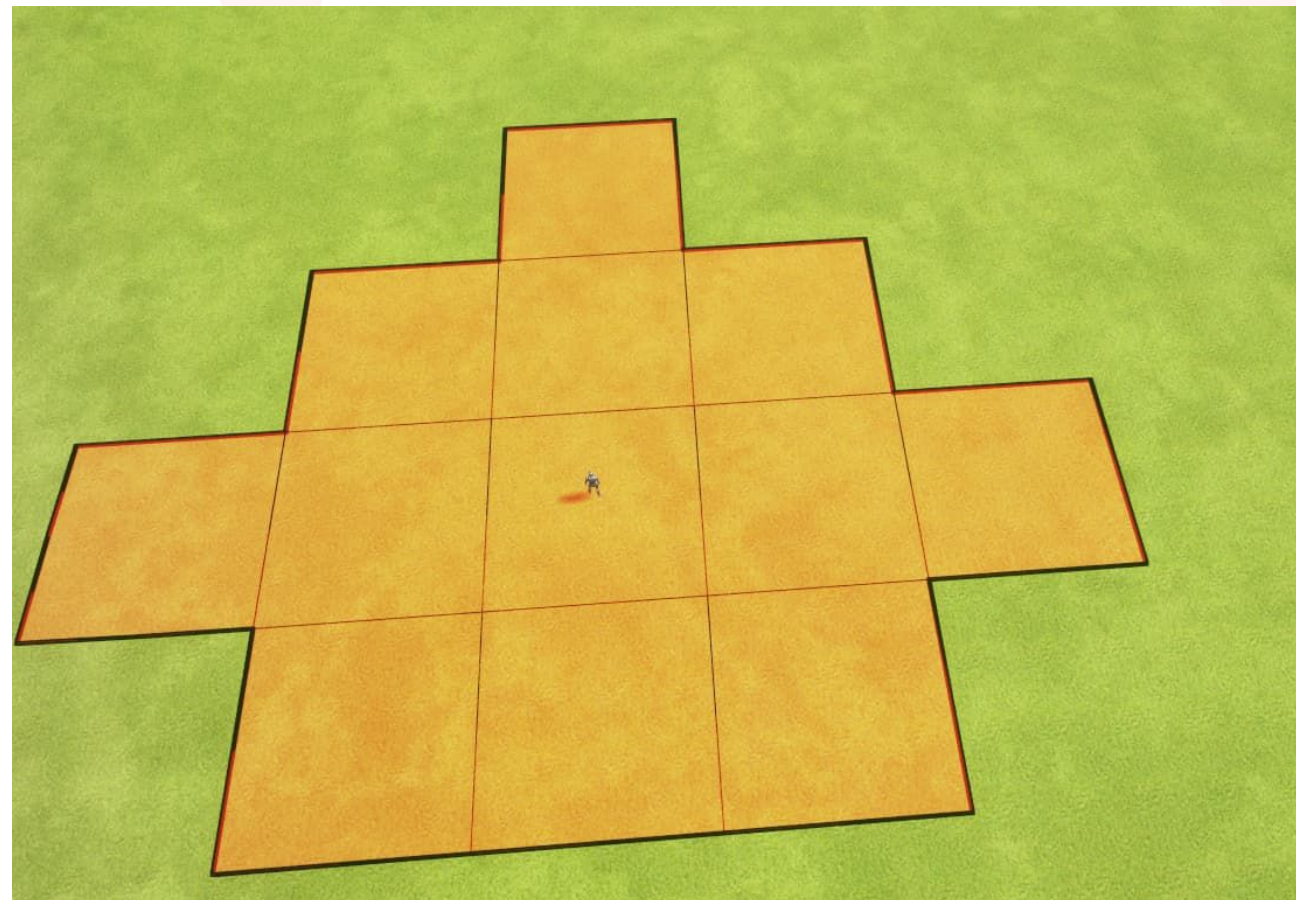


Nav Mesh Generation

In situazioni dove il mondo di gioco è molto ampio, come ad esempio gli *open world*, ambo i sistemi di generazione della navmesh possono essere **molto costosi**, in quanto la generazione della navmesh sia statica, quindi caricata offline all'inizio del gioco, che quella dinamica, generata a runtime, finisce per richiedere, probabilmente, un tempo considerevole.

Una possibile soluzione è l'utilizzo dei **Navigation Invoker**, un componente da associare ad un agente AI il quale genera navmesh solo intorno a se stesso, il tutto a runtime.

Questo sistema elimina la necessità di calcolare la mesh in anticipo o di doverne calcolare grosse quantità durante il gameplay, in quanto il sistema genera sì la mesh in tempo reale, ma solo in uno spazio limitato intorno all'attore, ottimizzando così le risorse e i tempi di calcolo.

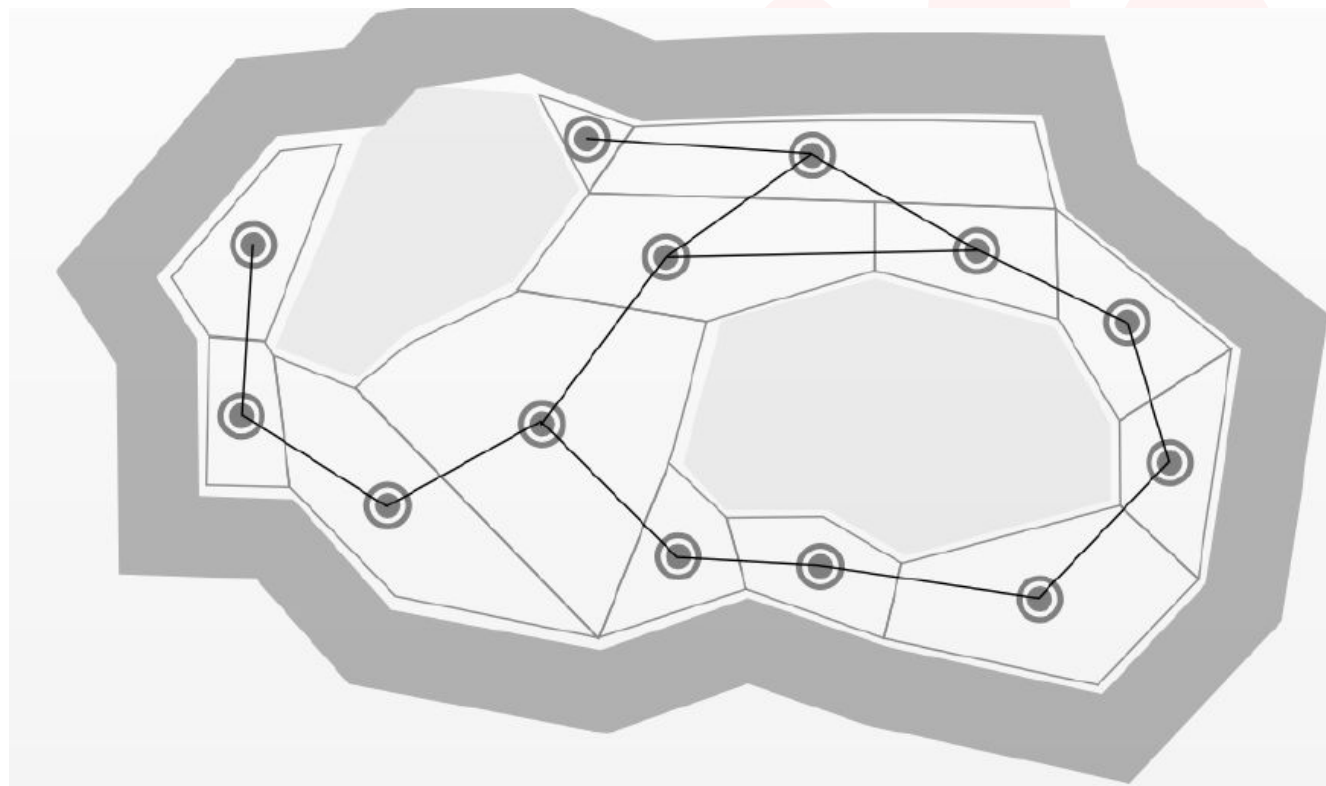




Nav Mesh Generation

Durante la fase di generazione delle navmesh viene generato un grafo che collega i diversi poligoni di cui la navmesh è composta.

Su questi si possono utilizzare algoritmi di pathfinding classici (su grafo) per trovare il percorsi più corto tra due aree, come fa UE tramite l'utilizzo di A*.



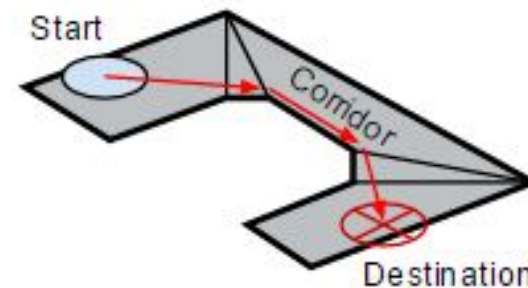
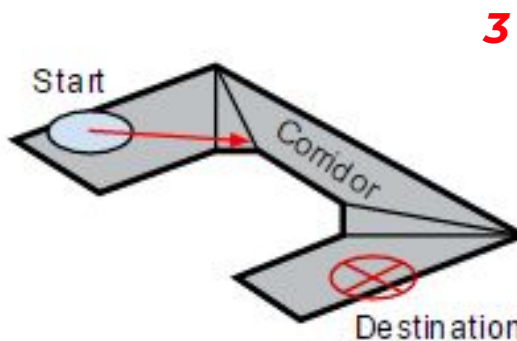
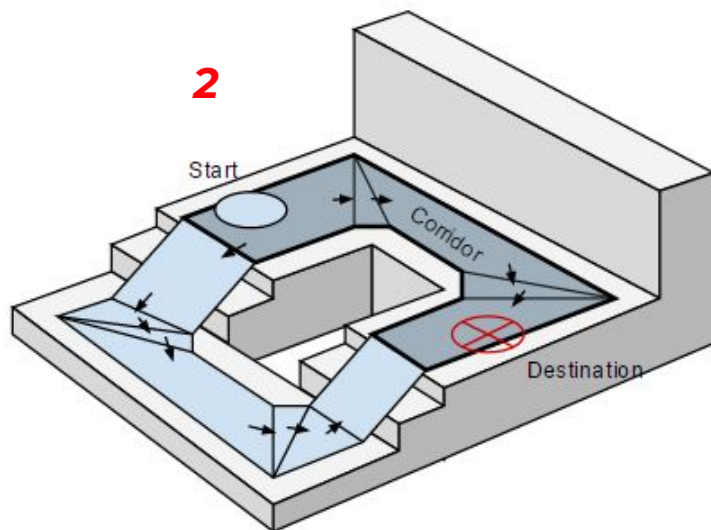
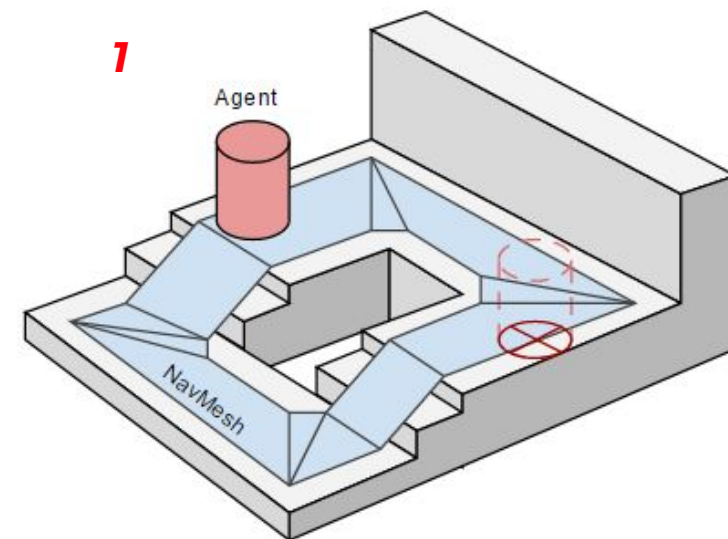


Nav Mesh Generation - Esempio

Dato un punto di partenza ed un punto finale si andranno a mappare i due punti sulle navmesh **(1)**.

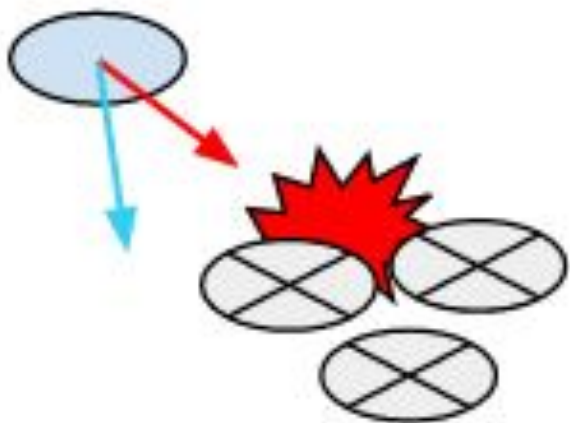
Successivamente si utilizzerà A* per trovare il percorso più corto **(2)**.

Infine, in fase di movimento, il NavMesh Agent seguirà il percorso così definito puntando sempre al vertice della navmesh visibile più vicino al percorso scelto **(3)**.





Steering

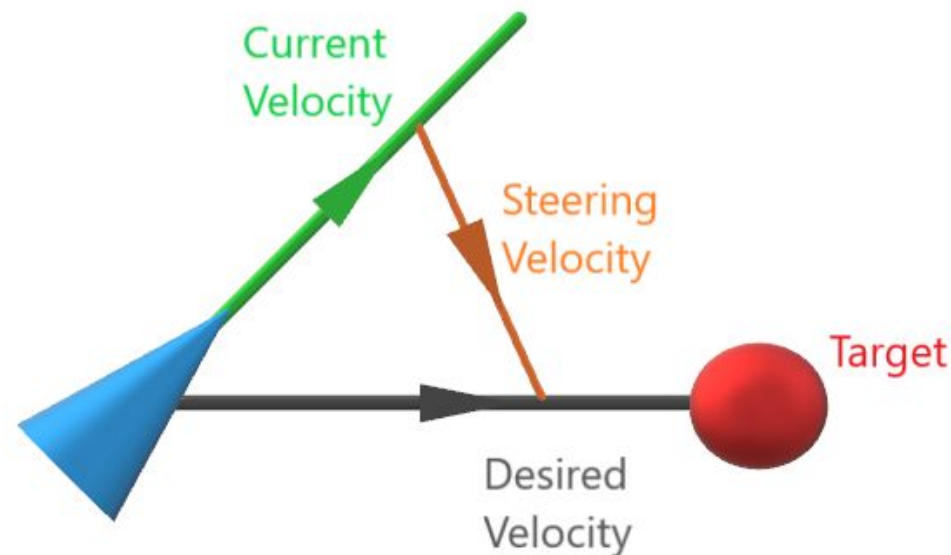


Per seguire i valori desiderati, un metodo base che viene utilizzato è un semplice modello dinamico, il quale considera il movimento come un processo continuo.

L'agente calcola forze dinamiche basate sull'ambiente circostante e sulle sue intenzioni, permettendoci di gestire veicoli o personaggi allo stesso modo.

In base al percorso scelto, l'engine genera una direzione e velocità desiderate per l'agente AI che cercherà di raggiungere, se possibile.

La determinazione di questi due valori è detto **steering** ed è fondamentale per permettere agli NPC di muoversi verso un obiettivo ed evitare gli ostacoli con movimenti il più possibile fluidi e naturali





Avoidance

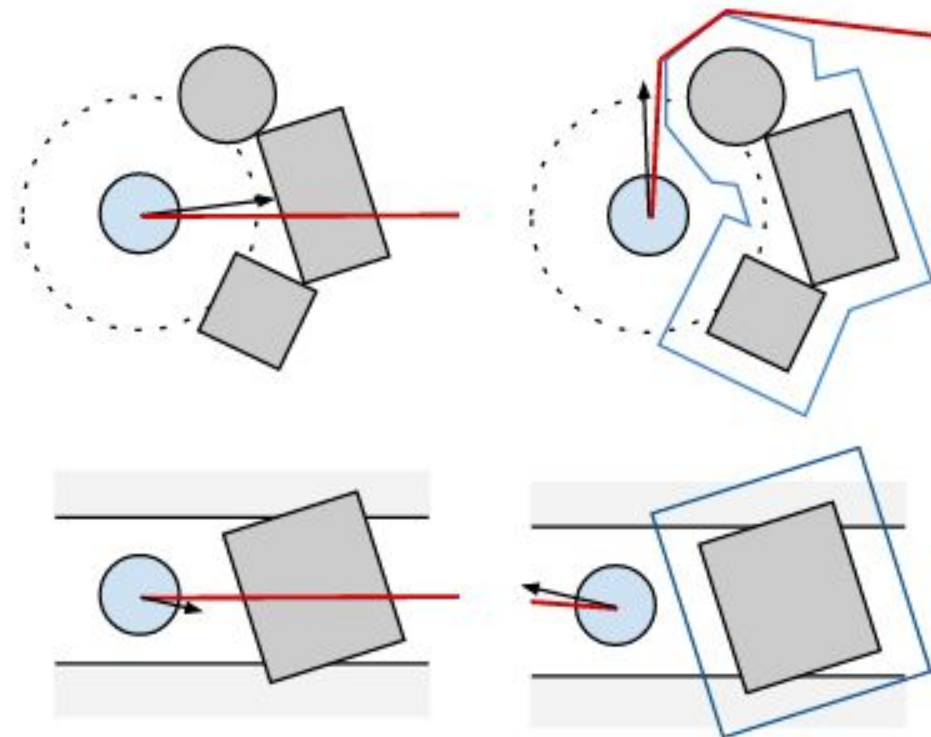
Quando però, ci si ritrova in presenza di ostacoli, una correzione ulteriore viene effettuata, chiamata **Obstacle Avoidance**.

Unreal Engine offre due sistemi di evitamento per prevenire collisioni tra entità in movimento:

Il sistema ***Reciprocal Velocity Obstacles*** (RVO), il quale calcola i vettori di velocità per ogni agente, tenendo conto degli agenti vicini e assumendo che si muovano a velocità costante ad ogni intervallo di calcolo. Il vettore di velocità ottimale scelto è quello che più si avvicina alla velocità desiderata dall'agente nella direzione della sua destinazione.

Questo sistema è integrato nel componente di movimento del personaggio (*Character Movement Component*).

L'RVO non utilizza la navmesh per l'evitamento degli ostacoli, quindi può essere usato indipendentemente dal Navigation System per qualsiasi personaggio.



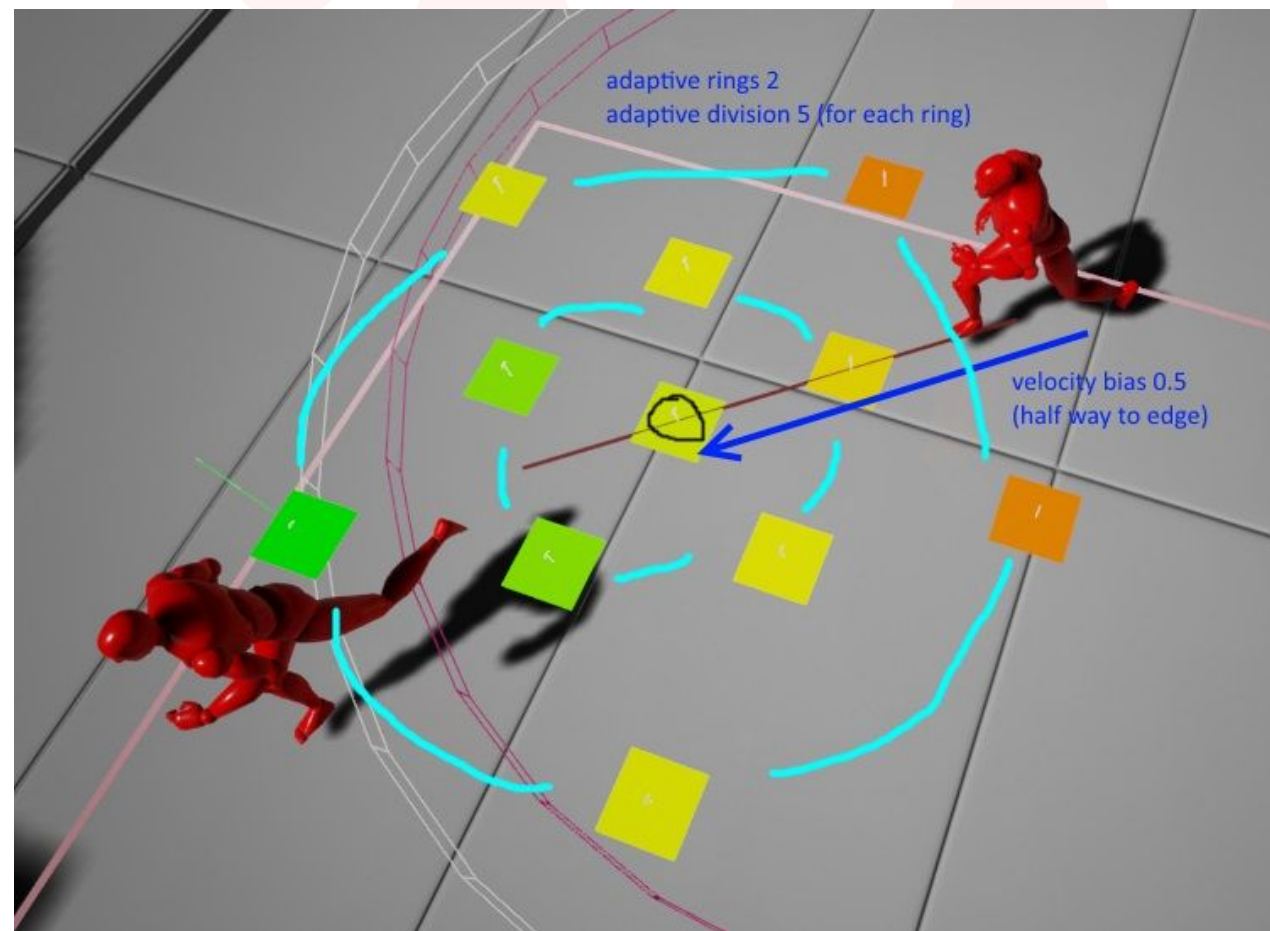


Avoidance

Il **Detour Crowd Manager** è un sistema avanzato di gestione della navigazione per gruppi di agenti AI che simula in modo realistico l'interazione e l'evitamento degli ostacoli tra più entità. È particolarmente utile simulare il movimento di gruppi di agenti in ambienti complessi.

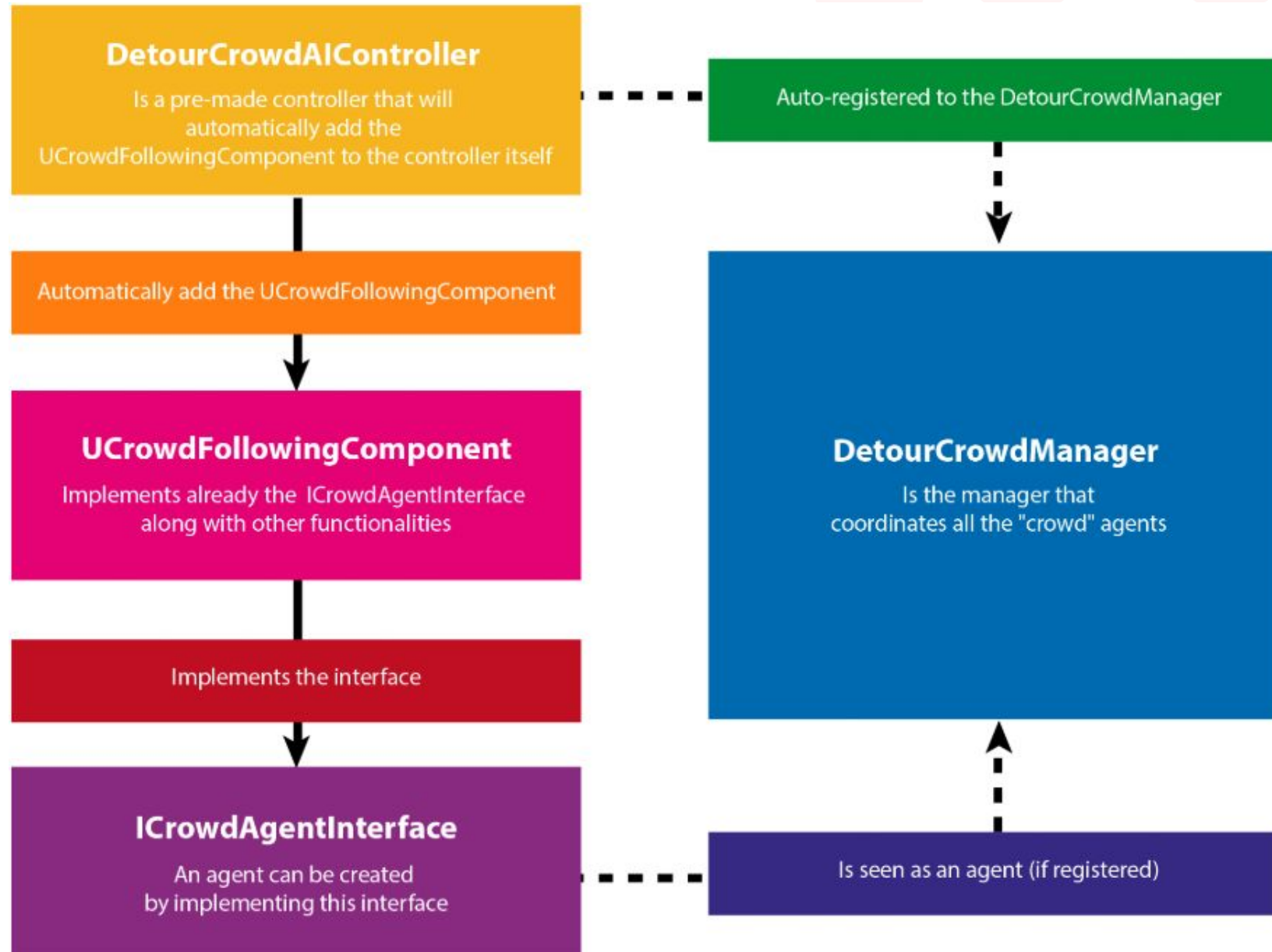
Utilizza un approccio che calcola velocità di movimento orientate verso la direzione dell'agente, ottimizzando l'evitamento degli ostacoli e migliorando il comportamento di folla. Rispetto al tradizionale sistema RVO, offre una qualità superiore nell'evitare le collisioni.

Il sistema è decisamente flessibile in quanto può essere utilizzato da qualsiasi attore che estende la classe *Pawn*, ed è una soluzione potente per creare movimenti realistici e coordinati tra NPC.





Detour Crowd System





Detour Crowd System - Setup

The screenshot shows the 'Project Settings' window for the Detour Crowd System. The left sidebar lists various settings categories, with 'Engine' and 'Crowd Manager' highlighted. The main panel displays the 'Engine - Crowd Manager' settings, which are saved in 'DefaultEngine.ini'. The 'Config' section is expanded, showing a list of settings for the AI Crowd Manager. A red arrow points from the 'Avoidance Config' section to a detailed inset window.

Game

- Asset Manager
- Engine**
- AI System
- Animation
- Audio
- Collision
- Console
- Cooker
- Crowd Manager
- Gameplay Debugger
- Garbage Collection
- General Settings
- Hierarchical LOD
- Input
- Navigation Mesh
- Navigation System
- Network
- Physics
- Rendering
- Rendering Overrides (Local)
- Slate Settings
- Streaming
- Tutorials
- User Interface

Engine - Crowd Manager

Settings for the AI Crowd Manager.

These settings are saved in DefaultEngine.ini, which is currently writable.

Config

- Avoidance Config: 4 Array elements
- Sampling Patterns: 0 Array elements
- Max Agents: 50
- Max Agent Radius: 100,0
- Max Avoided Agents: 6
- Max Avoided Walls: 8
- Navmesh Check Interval: 1,0
- Path Optimization Interval: 0,5
- Separation Dir Clamp: -1,0
- Path Offset Radius Multiplier: 1,0
- Resolve Collisions: ☐

Config

- Avoidance Config: 4 elements
- 0: 10 members
- Velocity Bias: 0.5
- Desired Velocity Weight: 2.0
- Current Velocity Weight: 0.75
- Side Bias Weight: 0.75
- Impact Time Weight: 2.5
- Impact Time Range: 2.5
- Custom Pattern Idx: 255
- Adaptive Divisions: 5
- Adaptive Rings: 2
- Adaptive Depth: 1
- 1: 10 members
- 2: 10 members
- 3: 10 members



Detour Crowd System - Setup 1

Max Agents: Questo è il numero massimo di agenti che il Crowd Manager gestirà. Ovviamente, più alto è il numero, più agenti è possibile posizionare contemporaneamente, ma ciò avrà un impatto sulle prestazioni. È importante pianificare attentamente quanti agenti sono necessari per il gioco. Inoltre, se si guarda al codice sorgente, questo numero verrà utilizzato per allocare la memoria necessaria al Crowd Manager per gestire gli agenti. È utile tenerne conto in situazioni in cui la memoria è limitata.

Max Agent Radius: Questo è il raggio massimo che un agente, che si è deviato dal Crowd Manager, può percorrere.

Max Avoided Agents: Questo è il numero massimo di agenti che il sistema Detour prende in considerazione, e viene anche chiamato "vicini". In altre parole, specifica quanti agenti vicini (massimo) devono essere presi in considerazione per il comportamento di evitamento.

Max Avoided Walls: Questo è il numero massimo di muri (in generale, segmenti di ostacolo) che il sistema Detour dovrebbe considerare. Funziona in modo simile a Max Avoided Agents, ma riguarda quanti segmenti di ostacoli circostanti devono essere presi in considerazione.

Navmesh Check Interval: Questo parametro definisce il numero di secondi che un agente che è uscito dalla navmesh dovrebbe aspettare prima di controllare e ricalcolare la sua posizione (il sistema tenterà di spingere l'agente indietro sulla navmesh).

Path Optimization Interval: Indica, in secondi, con quale frequenza un agente dovrebbe tentare di ottimizzare nuovamente il proprio percorso.

Separation Dir Clamp: Quando un altro agente si trova dietro, questo valore indica la forza di separazione limitata a sinistra/destra (prodotto scalare tra la direzione di avanzamento e la direzione verso il vicino; quindi, un valore di -1 significa che questo comportamento di separazione è disabilitato).

Path Offset Radius Multiplier: Quando l'agente sta girando vicino a un angolo, viene applicato un offset al percorso. Questa variabile è un moltiplicatore per tale offset (in modo che tu possa ridurre o aumentare l'offset a piacere).

Resolve Collisions: Nonostante i migliori sforzi del sistema Detour, gli agenti potrebbero comunque collidere. In tal caso, la collisione dovrebbe essere gestita dal sistema Detour (con il valore true per questa variabile). Se questa variabile è impostata su false, gli agenti utilizzeranno un Character Movement Component. Questo componente si occuperà di risolvere la collisione.



Detour Crowd System - Setup 2

Avoidance Config è un array di diverse configurazioni di campionamento, ciascuna con parametri leggermente differenti. Di default, ce ne sono quattro, corrispondenti a diversi livelli di qualità dell'avoidance: Basso, Medio, Buono e Alto.

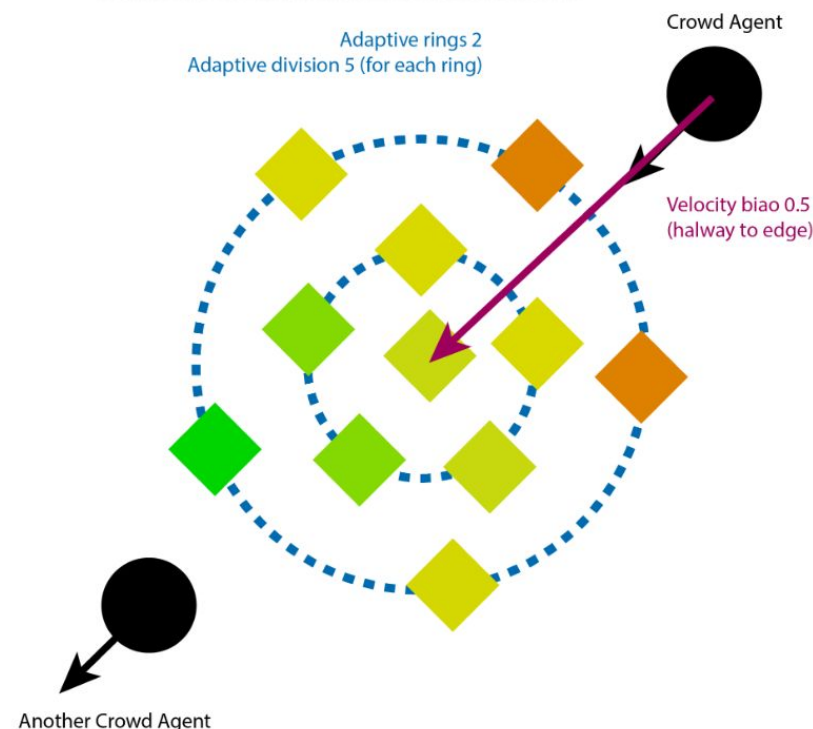
Il livello di qualità viene impostato nel **UCrowdFollowingComponent** tramite la variabile **AvoidanceQuality**, che utilizza l'enumerazione **ECrowdAvoidanceQuality**.

Se si ha un riferimento al **UCrowdFollowingComponent**, è possibile utilizzare la funzione `SetCrowdAvoidanceQuality()` per modificarlo.

La parte dell'algoritmo che si occupa del campionamento inizia creando un set di anelli (il numero è indicato dal parametro **Adaptive Rings**) intorno al punto centrale (dove si trova inizialmente l'agente, con un bias, dovuto al parametro **Velocity Bias**, nella direzione della velocità). Ognuno di questi anelli viene campionato (diviso) dalla **Adaptive Division**. Successivamente, l'algoritmo affina ricorsivamente la ricerca utilizzando un set di anelli più piccoli, centrati sul miglior campione ottenuto nell'iterazione precedente. Il processo viene ripetuto per **Adaptive Depth** volte.

Ad ogni iterazione, viene scelto il miglior campione prendendo in considerazione i seguenti fattori, con i vari parametri che determinano il **peso** (quanto è importante la considerazione rispetto alle altre):

- La direzione dell'agente corrisponde alla velocità attuale?
Il peso è determinato dal parametro **DesiredVelocityWeight**.
- L'agente si muove lateralmente?
Il peso è determinato dal parametro **SideBiasWeight**.
- L'agente collide con qualche ostacolo noto?
Il peso è determinato dal parametro **ImpactTimeWeight** (viene eseguita una scansione di un intervallo considerando la velocità attuale dell'agente, se collide usando tale velocità entro un intervallo di **ImpactTimeRange** secondi).





Avoidance - Dimostrazione





Navigation System - Esercizio

Creiamo il nostro *NPC* ed il suo *AI Controller*.

Creiamo dei figli (*child*) e differenziamoli tra loro (es. Colore della *Skeletal Mesh*)

Successivamente andiamo a creare un livello (*ThirdPerson*) aggiungendo la nostra *Nav Mesh* così da renderla navigabile per le AI.

Ora seguiamo a step:

1. Mettiamo un NPC da un lato e il Player dal lato opposto e semplicemente vedremo, andando in play, come questa segua una linea dritta per arrivare a noi (inseriamo, poi, anche degli ostacoli).
2. Aggiungiamo mano a mano i vari *Nav Modifier* per modificare i path e vediamo come si comporta l'NPC.

In aggiunta, andiamo a creare un custom *Nav Area Class* per provare a bilanciare quanto convenga all'NPC scegliere una strada più lunga, ma più leggera, rispetto ad una più corta e pesante.

3. Proviamo a mettere più NPC in scena con la stessa destinazione e vedere come si comportano. Successivamente proviamo ad abilitare, nello stesso percorso, prima l'RVO e poi il Crowd e vedere la differenza per quanto riguarda l'avoidance.
4. In ultimo andiamo a creare dei *Nav Query Filters* così da differenziare il terreno per vedere come ogni NPC si muoverà in base a quello selezionato per raggiungere al costo minore la sua meta finale.



Navigation System - Esercizio

Prendiamo il nostro *NPC* ed il suo *AI Controller*.

Successivamente andiamo a creare un livello con la nostra *Nav Mesh* che sia composto da waypoint in modo tale che possano essere raggiunti in modo consequenziale e che, quindi, tracceranno il percorso che le nostre *AI* dovranno percorrere per arrivare dal punto di partenza fino al rifugio finale.

Aggiungiamo *Nav Modifier* per modificare i path, così come dislivelli per utilizzare i *Nav Link*.

In ultimo andiamo ad utilizzare dei *Nav Query Filters* così da differenziare il terreno per vedere come ogni *NPC* si muoverà in base a quello selezionato per raggiungere al costo minore la sua meta finale.

Per renderlo più particolare, possiamo far sì che il nostro personaggio abbia una *Stamina* e che al calare di questa, determinati percorsi aumentino il loro costo, spingendo l'*NPC* a cambiare le sue scelte in corso d'opera.

